

VENTUREIQ

AI-Powered Multi-Agent Startup Due Diligence Engine with RAG & Live Market Intelligence

Project Title: VentureIQ Due Diligence Engine	Target Evaluation: College Viva & Project Defense
Architecture: Stateful LangGraph + Pinecone RAG	Tech Stack: React 19, FastAPI, Gemini, Tavily
Author/Developer: Student Project Submission	Date: September 2026

1. Executive Overview & Problem Statement

VentureIQ is an advanced, production-ready AI due diligence engine designed to automate the evaluation of startup concepts. Traditional startup assessment relies on manual market research, fragmented secondary sources, and static pitch-deck analysis, which are often prone to human bias, missing competitor data, and high latency.

Core Challenges Addressed:

- **LLM Hallucinations:** Standard single-prompt models frequently fabricate market size numbers, growth rates, and financial metrics.
- **Stale Knowledge Cutoffs:** Off-the-shelf generative models have no awareness of newly launched competitors or current market shifts.
- **Unstructured Reasoning:** A single LLM call lacks specialized domain focus (e.g., market sizing vs competitor analysis vs unit economics).
- **Lack of Verifiable Grounding:** Pitch evaluations require historical domain benchmark evidence and live web confirmation.

The VentureIQ Solution:

VentureIQ resolves these limitations by introducing a **Stateful Multi-Agent Graph (LangGraph)** augmented with a **Vector Retrieval-Augmented Generation (RAG) pipeline via Pinecone** and **Live Web Research via Tavily API**. The system breaks the evaluation into specialized autonomous agents (Planner, Market Research, Competitor Analysis, Business Model), each operating under strict schema constraints to produce factual, verifiable, and comprehensive investment reports.

2. Comprehensive System Architecture

The VentureIQ platform is organized into four distinct architectural layers, ensuring strict separation of concerns, high throughput, and seamless frontend state sync:

Layer	Component	Technical Responsibility & Implementation
Frontend Layer	React 19 + Vite 7 Three.js R3F Tailwind CSS	Provides a high-speed Single Page Application (SPA). Features an interactive 3D Intelligence Core rendering real-time node state transitions, live SSE log streaming, and formatted markdown report views.

API & Backend Layer	FastAPI Uvicorn ASGI Async Locks	Exposes REST endpoints (/api/generate-plan, /api/rag/ingest). Implements request deduplication locks to prevent redundant graph executions and handles asynchronous task orchestration.
Orchestration Engine	LangGraph LangChain Core StateGraph	Executes a stateful Directed Acyclic Graph (DAG). Manages agent context flow, parallel execution nodes, and node state transitions across Planner, Market, Competitor, and Business agents.
Data & RAG Layer	Pinecone Vector DB Gemini Embeddings Tavily Search API	text-embedding-004 converts startup benchmarks into 768-dim vector embeddings. Pinecone indexes cosine similarity matches. Tavily provides real-time web search for active market data.

3. LangGraph & RAG Detailed Execution Workflow

The end-to-end processing pipeline executes deterministically through six sequential stages:

Step 1: Input Ingestion & Sanitization

The user submits a startup idea (e.g., 'AI-powered pitch deck validator for VC firms'). The FastAPI server validates the input payload against Pydantic models.

Step 2: Vector RAG Evidence Retrieval (Pinecone)

Before engaging LLM agents, backend/rag/retriever.py generates a 768-dimensional embedding of the query using Google Gemini Embeddings. It queries Pinecone namespace venture-iq-knowledge to fetch top-k relevant historical market benchmarks, SaaS metrics, and risk factors.

Step 3: Planner Agent Sizing & Scope Definition

The Planner Node receives the user prompt merged with RAG vector evidence. It formulates a structured execution strategy, defining research hypotheses, target market segments, and specific search directives.

Step 4: Market Research Agent (Tavily Live Web Search)

The Market Agent executes external searches via Tavily API to gather live TAM, SAM, SOM estimates, CAGR projections, and market drivers. It synthesizes quantitative market figures grounded in real-time data.

Step 5: Competitor Analysis Agent

Searches for direct, indirect, and potential market entrants. Builds a detailed competitive matrix including SWOT analysis, key differentiators, pricing models, and defensive moats.

Step 6: Business Model & Synthesis Agent

Evaluates unit economics (CAC, LTV ratios), monetization channels, go-to-market strategy, regulatory compliance risks, and final investment risk scoring. Serializes the final output into clean JSON structure.

4. Technology Stack Selection & Engineering Rationale

Each technology in VentureIQ was selected after evaluating performance, maintainability, cost, and alignment with modern AI application standards:

Technology	Selected Tool	Engineering Rationale & Advantages over Alternatives
------------	---------------	--

Frontend Framework	React 19 + Vite 7	Vite provides instant HMR and ES module bundling. React 19's concurrent rendering allows smooth updates of live multi-agent streaming status without blocking UI interactions.
3D Graphics	Three.js / React Three Fiber	Renders an interactive 3D core representing agent nodes and data signals, transforming a static dashboard into an engaging presentation visual.
Backend API	FastAPI (Python 3.11+)	Built on Starlette and Pydantic. Provides native async support, automatic OpenAPI documentation, and effortless integration with Python AI packages.
Agent Framework	LangGraph	Unlike unstructured AutoGen or monolithic LangChain chains, LangGraph allows stateful cyclic and acyclic graph execution with strict state schemas and full control over agent memory.
Vector Database	Pinecone Vector DB	Fully managed serverless vector database providing ultra-fast sub-50ms vector searches, metadata filtering, and effortless scalability without infrastructure maintenance overhead.
LLM & Embeddings	Google Gemini 2.5 / 3.6 & Gemini Embeddings	Offers state-of-the-art context windows, superior JSON mode adherence, fast token generation speeds, and high quality 768-dimensional text embeddings.
Live Web Search	Tavily API	Specifically engineered for AI agents. Filters out web clutter, advertisements, and raw HTML, returning clean markdown content optimized for RAG consumption.

5. Competitive Superiority vs. Generic LLMs & ChatGPT

A common viva question is: *'Why build VentureIQ when someone can just ask ChatGPT?'* The comparison table below details the architectural advantages of VentureIQ over off-the-shelf single-prompt AI chatbots:

Dimension	Generic Chatbots (ChatGPT / Claude)	VentureIQ Engine
Architecture	Single-prompt monolith. One prompt attempts to answer all questions sequentially.	Multi-Agent Orchestration. Specialized graph nodes analyze distinct dimensions with focused prompts.
Market Sizing & Metrics	Static or hallucinated TAM/CAGR figures based on training text.	Live Tavily Web Verification. Fetches real-time financial stats, recent reports, and active market data.
Domain Benchmark Grounding	No access to private startup benchmark knowledge bases.	Pinecone Vector RAG. Queries specialized vector database for historical venture benchmarks and failure modes.
Factuality & Reliability	High risk of ungrounded statements and optimistic assumptions.	Cross-Verification Pipeline. Agent state validates facts across Pinecone RAG and live web search.
Output Structure	Unstructured chat text; non-standardized formats across queries.	Strict JSON Schema. Structured, predictable output feedable into dashboards, charts, and 3D visualizers.
Visual Experience	Plain text conversation window.	3D Intelligence Core & Real-time Graph Visualizer showing active node execution states dynamically.

6. Viva Voce Defense Questions & Technical Answers

Below are 15 technical viva questions prepared for project examination, covering theoretical concepts, implementation details, and trade-off choices:

Q1: What is Retrieval-Augmented Generation (RAG) and how is it implemented in VentureIQ?

RAG combines vector search retrieval with generative LLMs. In VentureIQ, when a user submits a startup idea, `backend/rag/retriever.py` uses Google Gemini Embeddings to generate a vector representation. It queries Pinecone vector database to retrieve relevant startup industry benchmarks and risk factors, injecting them into the LLM prompt context to eliminate hallucinations.

Q2: Why use LangGraph instead of standard LangChain linear chains or AutoGen?

Standard LangChain sequential chains are linear and rigid. AutoGen agents can loop unpredictably. LangGraph provides a stateful Directed Acyclic Graph (DAG) framework where each agent node modifies a shared global state dictionary. This allows deterministic execution, explicit node state tracking, error recovery, and clean UI state synchronization.

Q3: How do you handle concurrency and prevent duplicate requests in FastAPI?

We implement an `AsyncLock` and execution state tracking dictionary in `backend/main.py`. If a request for a specific session or concept is already in flight, duplicate incoming requests are safely rejected or queued, preventing race conditions and resource waste.

Q4: Which vector similarity metric is used in Pinecone and why?

We use **Cosine Similarity** (or Dot Product on normalized embeddings). Cosine similarity measures the angle between vectors regardless of magnitude, making it ideal for semantic text embeddings where text length variations shouldn't distort semantic similarity.

Q5: How does Tavily Web Search differ from standard web scraping with BeautifulSoup?

Standard web scraping retrieves raw HTML filled with ads, JavaScript scripts, and DOM clutter, requiring heavy parsing. Tavily is an LLM-native search API that executes domain-specific queries, extracts relevant body copy, strips non-essential text, and returns structured markdown optimized for direct prompt context injection.

Q6: How do you guarantee that the multi-agent system outputs structured data instead of unstructured text?

We enforce **Pydantic schemas** and use Gemini's structured output mode (JSON schema enforcement). The LLM is forced to format its response according to predefined JSON keys (e.g., TAM, SAM, SOM, SWOT array), which the backend validates before passing to the next agent.

Q7: What embedding model is used and what is its dimension size?

We use Google's text-embedding-004 (or embedding-001), generating **768-dimensional dense vector embeddings**. This embedding dimension balances semantic granularity with search performance in Pinecone.

Q8: How does the system handle API rate limits or network failures during Tavily or Gemini calls?

The workflow includes retry logic with exponential backoff. If an agent node encounters a network exception, the node catches the error, logs it to the state dictionary, and falls back to cached RAG evidence or structured fallback values without crashing the entire graph pipeline.

Q9: Explain the role of the 3D Intelligence Core in the frontend.

Built with Three.js and React Three Fiber, the 3D Core provides visual telemetry of agent operations. It dynamically updates node colors, rotation speeds, and glowing particle connections based on which agent (Planner, Market, Competitor, Business) is currently active in the backend stream.

Q10: What is the state schema managed by LangGraph during execution?

The global graph state contains keys: user_prompt, rag_context, plan, market_research, competitor_analysis, business_model, and logs. Each agent node reads prior state attributes and appends its output to the state.

Q11: Why is Pinecone vector database hosted serverless rather than locally (e.g., ChromaDB or FAISS)?

Serverless Pinecone provides enterprise-grade zero-maintenance persistence, instant horizontal scaling, low latency cloud API access, and eliminates local memory overhead during production deployment.

Q12: How do you measure the accuracy and quality of the generated due diligence report?

Evaluation is performed through groundedness checking: verifying that market statistics match live Tavily search citations and RAG retrieved document vectors, preventing ungrounded LLM assertions.

Q13: Can VentureIQ operate without internet connectivity?

The RAG component with local embeddings can run on pre-indexed Pinecone vectors, but live market research requires active web access via Tavily API to fetch current market metrics.

Q14: How are environment variables and secret keys handled securely?

Secrets (GEMINI_API_KEY, PINECONE_API_KEY, TAVILY_API_KEY) are loaded via python-dotenv from a single .env file. They are never hardcoded or logged in client-side bundles.

Q15: What are the primary future expansions planned for VentureIQ?

Key roadmap additions include multi-modal pitch deck PDF ingestion (OCR + vision model analysis), automated financial model generation (Excel output), and real-time founder audio pitch evaluation.

7. Step-by-Step Demonstration & Presentation Script

Use this structured 5-minute walkthrough script when presenting VentureIQ to professors or project evaluators:

Minute 0:00 - 1:00: Introduction & Problem Statement

- Start the application: Show Vite frontend running on `http://localhost:5173` and FastAPI on `http://localhost:8000`.
- Explain: 'Good morning Professors. Today we present VentureIQ, an AI multi-agent due diligence engine. Traditional startup analysis is manual, slow, and prone to LLM hallucinations. VentureIQ automates this using stateful agents, vector RAG, and live web research.'

Minute 1:00 - 2:00: Input Submission & RAG Execution

- Enter a startup concept into the prompt box (e.g. '*Autonomous AI Agent for B2B Invoice Auditing*') and click 'Analyze Startup'.
- Highlight: Point to the terminal logs and live UI stream showing **Pinecone RAG Vector Search** executing first, retrieving historic SaaS benchmark context.

Minute 2:00 - 3:30: LangGraph Multi-Agent Visual Execution

- Show the **3D Intelligence Core** active on screen.
- Point out the sequential state transitions:
 1. **Planner Agent**: Constructs research directives.
 2. **Market Agent**: Performs live Tavily web search, displaying real TAM/CAGR stats.
 3. **Competitor Agent**: Generates rival comparison matrix and SWOT.
 4. **Business Agent**: Computes unit economics and risk scoring.

Minute 3:30 - 4:30: Comprehensive Due Diligence Report Review

- Scroll through the rendered report tabs (Market Analysis, Competitors, Business Model, Risk Assessment).
- Show the factual citations and structured JSON rendering.

Minute 4:30 - 5:00: Conclusion & Technical Q&A; Ready

- Conclude: 'In summary, VentureIQ bridges the gap between raw LLM generation and enterprise-grade financial due diligence. We are now open for your questions.'

End of Master Guide — VentureIQ Presentation Ready